AI工具使用详情

1. 豆包

1.1 辅助润色模型假设

- 交互记录

A题.pdf
PDF · 292KB · 约 0.2 万字

建模前进行假设，如何描述云团互不影响

分享 ···

- **工程实际简化**：烟幕干扰弹的投放策略设计中，首要目标是“覆盖导弹飞行轨迹与真目标之间的空域”，若考虑云团交互（如扩散叠加、气流干扰），需引入流体力学、化学扩散等复杂模型，超出题目“投放策略优化”的核心需求，且现有参数（如扩散系数、气流速度）不足；
- 具体采纳：
  - **烟幕相互作用假设**：不同爆炸形成的烟幕云团之间不发生化学或流体动力学上的复杂交互，其遮蔽效果在空间上直接取并集[2]。

1.2 辅助转化excel为标准Latex格式

- 交互记录

对比维度	蒙特卡洛	PSO	单位
速度	101.15	70	m/s
方向	6	3.887	角度°
投弹时间	0.39	1.776551	s
爆炸延时	0.39	0	s
烟雾弹抛下位置x轴	17839.2324	17924.0725	m
烟雾弹抛下位置y轴	4.12349108	8.43001572	m
烟雾弹抛下位置z轴	1800	1800	m
爆炸位置x轴	17878.4648	17924.0725	m
爆炸位置y轴	8.24698217	8.43001572	m
爆炸位置z轴	1799.25471	1800	m
遮蔽时间	4.565	4.575	s
步长	0.05	0.05	\

请把这个表格写成overleaf格式

代码说明

- 使用 `tabular` 环境创建表格，`|l|r|r|l|` 表示表格有 4 列，列格式分别为左对齐（l）、右对齐（r）、右对齐（r）、左对齐（l），且列之间有竖线分隔。
- `\hline` 用于绘制水平分隔线，`\textbf{}` 用于设置表头文字为加粗格式。
- `\centering` 使表格在页面中居中显示，`\caption` 为表格添加标题，`\label` 用于给表格设置引用标签，方便在文档其他位置引用该表格。
- 若不需要中文支持，可移除 `\usepackage{ctex}` 包，同时将表格中的中文（如 “对比维度” 等）改为英文即可。

编辑 分享 ···

- 具体采纳

表 1　PSO 与蒙特卡洛算法结果对比

对比维度	蒙特卡洛算法	粒子群优化算法
速度 (m/s)	101.15	70
方向 (°)	6	3.887
投弹时间 (s)	0.39	1.776551
爆炸延时 (s)	0.39	0
遮蔽时间 (s)	**4.565**	**4.575**

2. deepseek（R1）

2.1 辅助修改代码使代码风格统一

- 交互记录

代码修改优化遮蔽计算

```
        print(f'遮蔽总时间:{effective_duration:.2f} s')

    return effective_duration


drone_speeds = [120, 70, 70, 70, 70]
drone_angles = [np.pi, 0, 0, 0, 0]
drone_flying_time = [1.5, 0, 0, 0, 0]
rest_time = [[1,1],[1,1],[1,1],[1,1],[1,1]]
bomb_burst_time = [[3.6, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0]]

particle = [drone_speeds, drone_angles, drone_flying_time, rest_time, bomb_burst_time]

total_cover_time = get_cover_time(particle, output=True)
```

请修改问题4的代码，使其与问题5代码风格（如宏定义、变量命名等）保持一致，同时注意修改问题4代码中未注意到“多颗烟雾弹可以同时遮蔽目标”这一问题，即问题5中涉及all_target_points_concealed的部分。

我将按照要求修改问题4的代码，使其与问题5的代码风格保持一致，并修正"多颗烟雾弹同时遮蔽目标"的问题。主要修改包括：

1. 添加宏定义常量（目标位置、导弹参数等）
2. 重构目标点生成方式（使用圆柱体模型）
3. 修正遮蔽判断逻辑（检查所有目标点是否被遮蔽）
4. 统一变量命名规范
5. 优化代码结构

修改后的代码：

```
import numpy as np

# 宏定义常量
FY_INITIAL = [
    np.array([17800, 0, 1800]),
    np.array([12000, 1400, 1400]),
    np.array([6000, -3000, 700])
]
M1_START = np.array([20000, 0, 2000])
MISSILE_TARGET_POINT = np.array([0, 0, 0])
TARGET_CYLINDER_BASE_CENTER = np.array([0, 200, 0])
TARGET_CYLINDER_RADIUS = 7.0
TARGET_CYLINDER_HEIGHT = 10.0
MISSILE_SPEED = 300.0
CLOUD_FALL_SPEED = 3.0
CLOUD_RADIUS = 10.0
```

- 具体采纳

  修改后在计算单个烟雾弹遮蔽时间时逻辑仍有错漏，顾采纳了风格建议，但仍手动修改了代码逻辑

3. ChatGPT

3.1 润色论文语言

- 交互记录

```
\subsubsection{分层优化策略}
由于解空间维度复杂且目标函数为"黑箱"，我们采用分层优化策略来求解。
\begin{itemize}
    \item \textbf{外层循环}：将无人机的飞行方向 $\theta$ 在可行域 $[0, \pi]$ 内进行离散化，分解为多个独立的子问题。
    \item \textbf{内层优化}：对于每个固定的 $\theta$ 值，问题被降维为求解 $(v, t_0, t_d)$ 三个参数的最优组合。
\end{itemize}
```

👌 我来帮你把 **“分层优化策略优势”** 这一部分写得更专业、更偏学术化。
你的原文重点是：**降维**、**减少局部最优风险**、**提高解释性**。我给你一个精炼升级版：

分层优化的优势

采用分层优化策略能够在以下几个方面显著提升求解效果：

1. **降维减复杂度**：原问题为四维优化空间 (θ, v, t_0, t_d)，直接全局搜索容易导致计算负担过大。通过将无人机飞行方向 θ 离散化，外层只需处理单变量枚举，内层则聚焦于三维连续优化，从而将高维复杂问题拆解为一系列低维子问题。
2. **降低陷入局部最优的风险**：在高维空间中，粒子群等群智能算法往往容易在复杂的搜索面上过早收敛。分层处理后，每个内层优化仅在较小的局部参数空间 (v, t_0, t_d) 内搜索，而外层离散化相当于在全局范围内进行“多起点搜索”。这种 **“全局分散 + 局部精细”** 的机制，能够有效避免算法停留在某一方向下的局部最优。
3. **提升全局覆盖性**：外层遍历 θ 值本质上保证了所有可能的飞行方向均被覆盖，相当于在方向维度上做了全局搜索；而内层则在该方向下做深度优化。这样就形成了 **全局–局部结合的优化框架**，既保证了解的全局性，又保留了局部优化的精细度。
4. **增强结果可解释性**：分层优化使得不同方向下的最优投放策略——对应，研究者可以直观比较不同方向的优劣，从而获得更清晰的物理意义解释，而不仅仅是“黑箱”最优解。

- 具体采纳 这种分层优化方法具有明显优势：首先，将四维参数空间（θ,v,t_0,t_d）拆分为“外层单变量离散化 + 内层三维连续优化”，有效缩小了单次优化的搜索维度，降低了粒子群在高维空间陷入局部最优的风险\cite{chatgpt}；其次，外层基于$ \theta $的规律性离散划分，使得每个子问题的优化目标更加集中（在固定方向下仅需调整投放参数），从而提升了优化结果的针对性与解释性。

4. Gemini

4.1 辅助生成可视化代码

- 交互记录

能不能给我一个py脚本画出局部最优解分布的三维图像等可视化图表

显示思路

当然可以。

要将一个8维的复杂解空间进行可视化，直接绘制一个三维图是不可能的。但是，我们可以通过一种有效的方法——**“切片分析”**，来观察解空间的局部形态。

思路如下：

1. **固定大部分变量：**我们先固定问题中8个变量中的6个，让它们取一个接近最优解的常量值。
2. **选择两个关键变量：**选择我们最感兴趣的两个变量作为坐标轴，例如**首枚投放时间 (t_drop1)** 和 **首枚引信时间 (t_fuse1)**，因为这两者对烟幕云的位置影响最为直接和耦合。
3. **绘制适应度曲面：**计算并绘制出在这两个变量变化时，“最大有效覆盖时间”（适应度）的变化情况。

这样得到的3D曲面图和2D等高线图，就是整个8维空间在特定位置的一个“切片”。图中的“山峰”就代

- 具体采纳

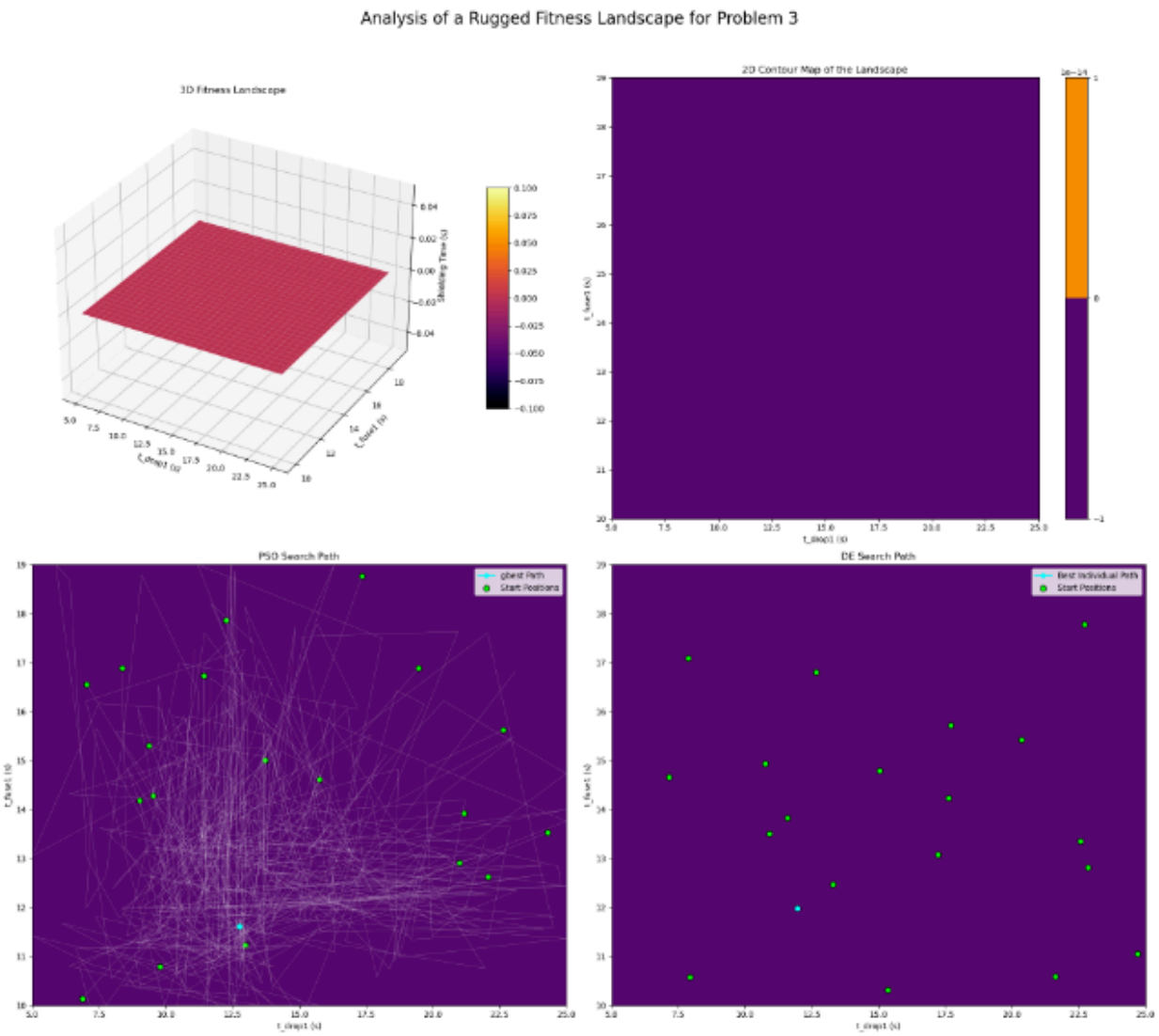

图 9　初期“零平台”及局部最优点分布

根据算法代码生成可视化图如上。